

# Informe de Desarrollo v0.0.2: Preprocesamiento de Espectros FTIR en Espacio de Absorbancia e Identificación Automática de Grupos Funcionales en GO y rGO

Matías Roberto Alemán

9 de junio de 2026

## Resumen

Este informe presenta el diseño matemático, algorítmico e ingenieril de un sistema automático de procesamiento y asignación de espectros de espectroscopía de infrarrojo por transformada de Fourier (FTIR) aplicado al Óxido de Grafeno (GO) y Óxido de Grafeno Reducido (rGO). Se describe la transición física del espacio de Transmitancia ( $%T$ ) al de Absorbancia ( $A$ ), un espacio linealmente aditivo según la ley de Beer-Lambert que mejora sustancialmente la robustez del filtrado. Mediante un barrido paramétrico exhaustivo de 180 configuraciones de preprocesamiento, se analizan los efectos del suavizado y la remoción de línea base, logrando la detección de la totalidad de las bandas analíticas clave. Finalmente, se presentan informes cuantitativos y una discusión fisicoquímica del proceso de reducción de los grupos oxigenados en la nanoestructura grafénica.

## Índice

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>1. Introducción y Fundamentos Físicos</b>                         | <b>3</b>  |
| <b>2. Transición al Espacio de Absorbancia</b>                       | <b>3</b>  |
| <b>3. Algoritmos de Suavizado</b>                                    | <b>3</b>  |
| 3.1. Promedio Móvil (Moving Average)                                 | 4         |
| 3.2. Filtro de Savitzky-Golay                                        | 4         |
| 3.3. Filtro de Mediana (Median Filter)                               | 5         |
| 3.4. Filtro de Percentil (Percentile Filter)                         | 5         |
| <b>4. Algoritmos de Corrección de Línea Base</b>                     | <b>6</b>  |
| 4.1. Eliminación de Tendencia Lineal (Detrend)                       | 6         |
| 4.2. Línea Base Lineal (Linear Baseline)                             | 6         |
| 4.3. Ajuste Polinomial Modificado Iterativo (IModPoly)               | 6         |
| 4.4. Mínimos Cuadrados Asimétricos (AsLS)                            | 6         |
| 4.5. Mínimos Cuadrados Penalizados Adaptativos (airPLS)              | 7         |
| 4.6. Mínimos Cuadrados Penalizados Asimétricos Locales (arPLS)       | 8         |
| <b>5. Búsqueda en Cuadrícula y Asignación de Grupos</b>              | <b>9</b>  |
| 5.1. Base de Datos de Grupos Funcionales                             | 9         |
| 5.2. Algoritmo de Búsqueda en Cuadrícula (Grid Search)               | 9         |
| <b>6. Algoritmos de Reportes y Comparación Espectral</b>             | <b>10</b> |
| 6.1. Detección Simultánea de Picos y Valles (Generación de Reportes) | 10        |
| 6.2. Superposición y Comparación de Espectros                        | 11        |
| 6.3. Mejora de Galería en Partes de $3 \times 1$ Subplots            | 11        |

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>7. Resultados Experimentales y Discusión Química</b>                       | <b>11</b> |
| 7.1. Resultados y Gráficos en Espacio de Transmitancia (%T) . . . . .         | 11        |
| 7.1.1. Preprocesamiento y Búsqueda en Cuadrícula para GO y rGO . . . . .      | 11        |
| 7.1.2. Galería de Espectros en Transmitancia . . . . .                        | 12        |
| 7.1.3. Superposición Comparativa en Transmitancia . . . . .                   | 12        |
| 7.2. Resultados y Gráficos en Espacio de Absorbancia (A) . . . . .            | 13        |
| 7.2.1. Preprocesamiento y Búsqueda en Cuadrícula en Absorbancia . . . . .     | 13        |
| 7.2.2. Galería Modular en Absorbancia (Lotes de 3x1) . . . . .                | 13        |
| 7.2.3. Superposición Comparativa y Discusión Química en Absorbancia . . . . . | 13        |
| <b>8. Manual de Usuario: Instalación y Recreación de Pruebas</b>              | <b>15</b> |
| 8.1. Instalación de la Biblioteca con Pip . . . . .                           | 15        |
| 8.2. Actualización de la Biblioteca . . . . .                                 | 15        |
| 8.3. Estructura del Proyecto y Recreación de Pruebas . . . . .                | 15        |
| <b>9. Conclusiones</b>                                                        | <b>16</b> |

## 1. Introducción y Fundamentos Físicos

El Óxido de Grafeno (GO) es un nanomaterial de carbono bidimensional caracterizado por una alta densidad de grupos funcionales que contienen oxígeno, tales como hidroxilos, epoxilos, carbonilos y carboxilos, los cuales decoran su plano basal y bordes. El proceso de reducción química o térmica del GO para producir Óxido de Grafeno Reducido (rGO) remueve la mayoría de estos grupos, restaurando la hibridación  $sp^2$  y la conjugación electrónica de la red de grafeno.

La espectroscopía FTIR es la técnica analítica de referencia para monitorear esta transición. Sin embargo, las señales crudas provenientes del espectrómetro contienen fuentes severas de distorsión:

1. **Ruido de alta frecuencia:** Proveniente de fluctuaciones del detector y del entorno.
2. **Deriva de línea base (Baseline drift):** Provocada por la dispersión física de la luz (Scattering de Mie y Rayleigh) y la absorción no específica del soporte.

Para corregir estas distorsiones y cuantificar los espectros de manera reproducible, se desarrolló una biblioteca algorítmica modular en Python que implementa filtros de suavizado avanzados y métodos iterativos de corrección de línea base penalizados.

## 2. Transición al Espacio de Absorbancia

Originalmente, los espectros FTIR se registran en Transmitancia por ciento ( $\%T$ ), definida como:

$$\%T = \frac{I}{I_0} \times 100 \quad (1)$$

Donde  $I$  es la intensidad transmitida a través de la muestra e  $I_0$  es la intensidad del blanco (background). En este espacio, los valles (mínimos locales) representan las bandas de absorción.

Sin embargo, la transmitancia no es linealmente proporcional a la concentración del analito ni posee propiedades aditivas. Para solventar esto, aplicamos la transformación logarítmica para obtener la **Absorbancia** ( $A$ ):

$$A = -\log_{10} \left( \frac{\%T}{100} \right) = 2 - \log_{10}(\%T) \quad (2)$$

La Absorbancia cumple la ley de Beer-Lambert, donde  $A = \epsilon \cdot c \cdot l$ , siendo  $\epsilon$  la absorptividad molar,  $c$  la concentración y  $l$  el camino óptico. En el espacio de Absorbancia:

- Las bandas de absorción apuntan hacia **arriba** (máximos locales).
- La línea base se ubica en el fondo de la señal (baja absorbancia).
- Las correcciones de línea base y las sustracciones de solvente son de naturaleza **aditiva** ( $A_{\text{corregida}} = A_{\text{medida}} - A_{\text{fondo}}$ ), lo que preserva matemáticamente la simetría y el área real de los picos.

Para prevenir indeterminaciones numéricas debido a valores de transmitancia nulos o ligeramente negativos (comunes en muestras muy absorbentes o zonas de alta absorción del solvente), la biblioteca implementa un recorte (clipping) preventivo de seguridad:

$$\%T_{\text{seguro}} = \text{clip}(\%T, 10^{-5}, 150, 0) \quad (3)$$

## 3. Algoritmos de Suavizado

El suavizado atenúa el ruido blanco instrumental. Se detallan a continuación las formulaciones y pseudocódigos de los cuatro métodos implementados.

### 3.1. Promedio Móvil (Moving Average)

Reemplaza cada elemento con el promedio de una ventana simétrica de tamaño  $W = 2k + 1$ :

$$A_{\text{smooth}}[i] = \frac{1}{2k+1} \sum_{j=-k}^k A[i+j] \quad (4)$$

Los extremos se manejan mediante acolchado (padding) por repetición del valor límite del espectro.

```
1 Algoritmo Promedio_Movil
2   Entrada: Vector y de longitud N, ventana W (impar)
3   Salida: Vector y_smooth
4
5   k = parte_entera(W / 2)
6   y_pad = Copiar y de longitud N + 2*k:
7       y_pad[0 a k-1] = y[0]
8       y_pad[k a N+k-1] = y
9       y_pad[N+k a N+2*k-1] = y[N-1]
10  y_smooth = Inicializar vector de longitud N con ceros
11  Para i desde 0 hasta N - 1 hacer:
12      Suma = 0
13      Para j desde -k hasta k hacer:
14          Suma = Suma + y_pad[i + k + j]
15      y_smooth[i] = Suma / W
16  Retornar y_smooth
17 Fin Algoritmo
```

Listado de código 1: Algoritmo de Promedio Móvil

### 3.2. Filtro de Savitzky-Golay

Propuesto originalmente por Savitzky y Golay en 1964 [1], realiza un ajuste local de mínimos cuadrados de un polinomio de grado  $d$  sobre una ventana de tamaño  $W = 2k + 1$ :

$$p_i(z) = \sum_{m=0}^d a_m z^m, \quad z \in [-k, k] \quad (5)$$

El valor suavizado es el término independiente:  $A_{\text{smooth}}[i] = p_i(0) = a_0$ . Este proceso equivale a una convolución con coeficientes fijos precalculados  $c_j$  mediante la pseudoinversa de la matriz de Vandermonde local  $J$ :

$$C = (J^T J)^{-1} J^T, \quad J_{z,m} = z^m \quad (6)$$

```
1 Algoritmo Savitzky_Golay
2   Entrada: Vector y de longitud N, ventana W (impar), grado del polinomio d (
3   d < W)
4   Salida: Vector y_smooth
5
6   k = parte_entera(W / 2)
7   Construir matriz J de tamaño (W x (d+1)):
8       Para cada z en [-k, k] y m en [0, d]:
9           J[z + k, m] = z^m
10  C = (J^T * J)^(-1) * J^T
11  coefs = primera fila de C
12  y_pad = Acolchar y en bordes usando simetria de espejo de longitud k
13  y_smooth = Inicializar vector de longitud N
14  Para i desde 0 hasta N - 1 hacer:
15      y_smooth[i] = Suma(coefs[j+k] * y_pad[i + j + k]) para j desde -k hasta
16      k
17  Retornar y_smooth
18 Fin Algoritmo
```

Listado de código 2: Algoritmo de Savitzky-Golay

### 3.3. Filtro de Mediana (Median Filter)

Filtro no lineal que remueve picos de ruido extremos (spikes) sin difuminar los bordes de los picos analíticos:

$$A_{\text{smooth}}[i] = \text{mediana}(\{A[i + j] \mid -k \leq j \leq k\}) \quad (7)$$

```
1 Algoritmo Filtro_Mediana
2   Entrada: Vector y de longitud N, ventana W (impar)
3   Salida: Vector y_smooth
4
5   k = parte_entera(W / 2)
6   y_smooth = Inicializar vector de longitud N
7   Para i desde 0 hasta N - 1 hacer:
8       ventana = subvector de y centrado en i de tamaño W (con relleno de
        borde)
9       Ordenar ventana de menor a mayor
10      y_smooth[i] = ventana[k]
11  Retornar y_smooth
12 Fin Algoritmo
```

Listado de código 3: Algoritmo de Filtro de Mediana

### 3.4. Filtro de Percentil (Percentile Filter)

Se selecciona el valor en el percentil  $P \in [0, 100]$ :

$$A_{\text{smooth}}[i] = \text{Percentil}_P(\{A[i + j] \mid -k \leq j \leq k\}) \quad (8)$$

```
1 Algoritmo Filtro_Percentil
2   Entrada: Vector y de longitud N, ventana W, percentil P (0 a 100)
3   Salida: Vector y_smooth
4
5   k = parte_entera(W / 2)
6   y_smooth = Inicializar vector de longitud N
7   Para i desde 0 hasta N - 1 hacer:
8       ventana = subvector de y centrado en i de tamaño W
9       Ordenar ventana de menor a mayor
10      indice_p = redondear((P / 100) * (W - 1))
11      y_smooth[i] = ventana[indice_p]
12  Retornar y_smooth
13 Fin Algoritmo
```

Listado de código 4: Algoritmo de Filtro de Percentil

## 4. Algoritmos de Corrección de Línea Base

En el espacio de Absorbancia, las bandas químicas apuntan hacia arriba. Esto significa que la línea base  $z$  se encuentra por debajo del espectro y debe seguir la envolvente inferior.

### 4.1. Eliminación de Tendencia Lineal (Detrend)

Ajusta por mínimos cuadrados una recta global  $z = mx + b$  y la sustrae:

$$A_{\text{corregido}} = A_{\text{medido}} - z \quad (9)$$

### 4.2. Línea Base Lineal (Linear Baseline)

Toma el primer y último punto del espectro para formar un segmento de interpolación lineal y lo sustrae.

### 4.3. Ajuste Polinomial Modificado Iterativo (IModPoly)

Ajusta iterativamente un polinomio  $z(x) = \sum_{j=0}^{\text{deg}} a_j x^j$ . Para evitar que el polinomio sea arrastrado hacia arriba por los picos de absorbancia, en cada iteración  $t$ , si el dato crudo supera la línea polinomial estimada más la desviación estándar de los residuos  $\sigma$ , el dato se reemplaza por el valor del polinomio en ese punto:

$$A^{(t)}[i] = \begin{cases} P_t(x_i) & \text{si } A[i] > P_t(x_i) + \sigma \\ A[i] & \text{de lo contrario} \end{cases} \quad (10)$$

```
1 Algoritmo IModPoly_Absorbancia
2   Entrada: Vector x, Vector y, grado d, iteraciones MaxIter
3   Salida: Línea base z
4
5   y_base = Copiar y
6   Para iter desde 1 hasta MaxIter hacer:
7       coefs = ajustar_polinomio(x, y_base, d)
8       z = evaluar(coefs, x)
9       sigma = desv_estandar(y - z)
10      Para i desde 0 hasta N - 1 hacer:
11          Si y[i] > z[i] + sigma entonces: (Picos apuntan hacia arriba en
absorbancia)
12              y_base[i] = z[i]
13          Sino:
14              y_base[i] = y[i]
15      coefs = ajustar_polinomio(x, y_base, d)
16      z = evaluar(coefs, x)
17      Retornar z
18 Fin Algoritmo
```

Listado de código 5: Algoritmo IModPoly (Absorbancia)

### 4.4. Mínimos Cuadrados Asimétricos (AsLS)

Desarrollado por Eilers [2], balancea la fidelidad a los datos con la suavidad de la línea base minimizando:

$$S = \sum_{i=1}^N w_i (A_i - z_i)^2 + \lambda \sum_{i=3}^N (\Delta^2 z_i)^2 \quad (11)$$

Donde  $\Delta^2$  es la segunda diferencia discreta y  $\lambda$  la penalización de suavidad. Los pesos se actualizan como:

$$w_i = \begin{cases} p & \text{si } A_i > z_i \\ 1 - p & \text{si } A_i \leq z_i \end{cases} \quad (12)$$

En forma matricial se resuelve mediante el sistema lineal disperso:

$$(W + \lambda D^T D)z = WA \quad (13)$$

```

1 Algoritmo AsLS_Absorbancia
2   Entrada: Vector y de longitud N, suavizado lam, asimetria p, iteraciones
   MaxIter
3   Salida: Linea base z
4
5   D2 = construir_matriz_diferencia_segundo_orden(N)
6   D = D2^T * D2
7   w = Vector de unos de longitud N
8   Para iter desde 1 hasta MaxIter hacer:
9       W = construir_matriz_diagonal(w)
10      z = resolver_sistema_lineal(W + lam * D, w * y)
11      Para i desde 0 hasta N - 1 hacer:
12          Si y[i] > z[i] entonces:
13              w[i] = p
14          Sino:
15              w[i] = 1 - p
16      Retornar z
17 Fin Algoritmo

```

Listado de código 6: Algoritmo AsLS (Absorbancia)

#### 4.5. Mínimos Cuadrados Penalizados Adaptativos (airPLS)

Introducido por Zhang et al. [3], airPLS estima de forma adaptativa los pesos sin requerir el parámetro  $p$ . Si en la iteración  $t$  el residuo es  $d_i = A_i - z_i$ , los pesos son:

$$w_i = \begin{cases} 0 & \text{si } A_i \geq z_i \\ \exp\left(\frac{t \cdot (A_i - z_i)}{\text{std}(d^-)}\right) & \text{si } A_i < z_i \end{cases} \quad (14)$$

```

1 Algoritmo airPLS_Absorbancia
2   Entrada: Vector y de longitud N, suavizado lam, iteraciones MaxIter,
   tolerancia tol
3   Salida: Linea base z
4
5   D2 = construir_matriz_diferencia_segundo_orden(N)
6   D = D2^T * D2
7   w = Vector de ones de longitud N
8   Para t desde 1 hasta MaxIter hacer:
9       W = construir_matriz_diagonal(w)
10      z = resolver_sistema_lineal(W + lam * D, w * y)
11      d = y - z
12      d_neg = subvector(d < 0)
13      Si d_neg esta vacio, romper ciclo.
14      std_neg = desv_estandar(d_neg)
15      Si std_neg < tol, romper ciclo.
16      Para i desde 0 hasta N - 1 hacer:
17          Si d[i] < 0 entonces:
18              w[i] = exp(t * d[i] / std_neg)
19          Sino:
20              w[i] = 0
21      Retornar z
22 Fin Algoritmo

```

Listado de código 7: Algoritmo airPLS (Absorbancia)

#### 4.6. Mínimos Cuadrados Penalizados Asimétricos Locales (arPLS)

Propuesto por Baek et al. [4], arPLS utiliza una función logística continua para actualizar los pesos:

$$w_i = \frac{1}{1 + \exp\left(\frac{d_i - (2\sigma^- + \mu^-)}{\sigma^-}\right)} \quad (15)$$

```
1 Algoritmo arPLS_Absorbancia
2   Entrada: Vector y de longitud N, suavizado lam, iteraciones MaxIter,
   tolerancia tol
3   Salida: Linea base z
4
5   D2 = construir_matriz_diferencia_segundo_orden(N)
6   D = D2^T * D2
7   w = Vector de ones de longitud N
8   Para iter desde 1 hasta MaxIter hacer:
9       W = construir_matriz_diagonal(w)
10      z = resolver_sistema_lineal(W + lam * D, w * y)
11      d = y - z
12      d_neg = subvector(d < 0)
13      Si d_neg esta vacio:
14          mean_neg = 0.0
15          std_neg = desv_estandar(d)
16      Sino:
17          mean_neg = promedio(d_neg)
18          std_neg = desv_estandar(d_neg)
19      Si std_neg < tol, romper ciclo.
20      Para i desde 0 hasta N - 1 hacer:
21          exponente = (d[i] - (2 * std_neg + mean_neg)) / std_neg
22          exponente = limitar_entre_valores(exponente, -50, 50)
23          w[i] = 1.0 / (1.0 + exp(exponente))
24   Retornar z
25 Fin Algoritmo
```

Listado de código 8: Algoritmo arPLS (Absorbancia)

## 5. Búsqueda en Cuadrícula y Asignación de Grupos

El espacio combinatorio se define como:

$$N_{\text{total}} = N_{\text{suavizados}} \times N_{\text{baselines}} = 12 \times 15 = 180 \text{ pruebas} \quad (16)$$

Los resultados se guardan en los archivos `absorbance_pipeline_results_rG0.csv` y `absorbance_pipeline_results_G0.csv`.

### 5.1. Base de Datos de Grupos Funcionales

Los intervalos de asignación de bandas FTIR para óxidos de grafeno son:

Tabla 1: Rangos espectrales analíticos en Absorbancia.

| Grupo Funcional       | Rango ( $\text{cm}^{-1}$ ) | Enlace y Modo Vibracional                   |
|-----------------------|----------------------------|---------------------------------------------|
| <b>O-H</b>            | [3200, 3600]               | Estiramiento O-H (alcohol, agua adsorbida)  |
| <b>C-H</b>            | [2800, 3000]               | Estiramiento C-H alifático ( $sp^3$ )       |
| <b>CO<sub>2</sub></b> | [2300, 2400]               | Dióxido de carbono atmosférico residual     |
| <b>C=O</b>            | [1650, 1750]               | Estiramiento C=O (carbonilo/carboxilo)      |
| <b>C=C</b>            | [1500, 1650]               | Estiramiento C=C del plano aromático $sp^2$ |
| <b>C-O</b>            | [1250, 1450]               | Estiramiento C-O carboxílico / bending C-OH |
| <b>C-O-C</b>          | [950, 1250]                | Estiramiento C-O-C de grupos epoxi          |

### 5.2. Algoritmo de Búsqueda en Cuadrícula (Grid Search)

Para identificar automáticamente la combinación óptima de filtrado y remoción de línea base, la biblioteca implementa una búsqueda en cuadrícula combinatoria. Para cada par de algoritmos de suavizado  $S$  y corrección de línea base  $B$ , el espectro preprocesado se somete a detección de picos y asignación a los grupos funcionales analíticos de la base de datos. Se calcula una puntuación de rendimiento basada en la cantidad de picos válidos detectados ( $N_v$ ), grupos funcionales mapeados ( $N_g$ ) y picos de ruido residuales ( $N_{\text{ruido}}$ ):

$$\text{Score} = N_v + N_g - w_n \cdot N_{\text{ruido}} \quad (17)$$

Donde  $w_n$  es el factor de penalización de ruido (normalmente  $w_n = 2,0$ ).

```

1 Algoritmo Busqueda_Cuadrícula
2   Entrada: Vector x, Vector y, Prominencia P, Distancia D, Peso w_n
3   Salida: Mejor_Config, Historial_Resultados
4
5   Historial_Resultados = Lista vacía
6   Para cada smooth_cfg en SMOOTHING_CONFIGS hacer:
7     y_smooth = aplicar_suavizado(y, smooth_cfg)
8     Para cada base_cfg en BASELINE_CONFIGS hacer:
9       Intentar:
10        z, y_corr = aplicar_linea_base(x, y_smooth, base_cfg)
11        picos_x, picos_y = detectar_picos(x, y_corr, P, D)
12        grupos_mapeados, ruido = asignar_grupos(picos_x)
13
14        score_std = calcular_score(picos_x, grupos_mapeados, ruido,
15        1.0)
16
17        score_pen = calcular_score(picos_x, grupos_mapeados, ruido, w_n
18        )
19
20        Registrar en Historial_Resultados:
21          smooth_cfg.label, base_cfg.label,
22          len(grupos_mapeados), len(picos_x), len(ruido), score_pen
23
24   Sino:
25     Ignorar configuracion fallida

```

```

22
23     Filtrar Historial_Resultados excluyendo Baseline == "RAW"
24     Mejor_Config = elemento en Historial con maximo(score_pen)
25     Retornar Mejor_Config, Historial_Resultados
26 Fin Algoritmo

```

Listado de código 9: Algoritmo Modular Combinatorio de Búsqueda en Cuadrícula

## 6. Algoritmos de Reportes y Comparación Espectral

Para facilitar el uso cuantitativo en laboratorios sin programadores avanzados, los módulos de análisis y visualización se estructuraron de forma totalmente modular en la biblioteca `ftir_library`.

### 6.1. Detección Simultánea de Picos y Valles (Generación de Reportes)

La caracterización física requiere extraer no solo los picos de absorción (máximos de absorbancia), sino también los valles o mínimos locales que delimitan las bandas. Para ello, el algoritmo calcula simultáneamente los extremos del espectro  $y(x)$  aplicando condiciones de vecindario local y prominencia mínima. Un pico a la frecuencia  $x_c$  se define si:

$$y_c > y_{c \pm j} \quad \forall j \in [1, D], \quad \text{y} \quad \text{prominencia}(y_c) \geq P_{\text{picos}} \quad (18)$$

Y un valle en  $x_v$  satisface:

$$y_v < y_{v \pm j} \quad \forall j \in [1, D], \quad \text{y} \quad \text{prominencia}(-y_v) \geq P_{\text{valles}} \quad (19)$$

```

1  Algoritmo Generar_Reporte_Numerico
2      Entrada: Vector x, Vector y_corr, Nombre_Muestra, Ruta_CSV, P_picos,
3      P_valles, D
4      Salida: Reporte en archivo CSV
5
6      picos_idx = buscar_maximos_locales(y_corr, prominencia=P_picos, distancia=D)
7
8      valles_idx = buscar_maximos_locales(-y_corr, prominencia=P_valles,
9      distancia=D)
10
11     reporte = Lista vacia
12     grupos_mapeados, _ = asignar_grupos(x[picos_idx])
13
14     Para cada idx en picos_idx hacer:
15         w = x[idx]
16         val = y_corr[idx]
17         g = buscar_grupo_asociado(w, grupos_mapeados)
18         reporte.agregar(Tipo="Peak", Wavenumber=w, Absorbance=val, Group=g)
19
20     Para cada idx en valles_idx hacer:
21         w = x[idx]
22         val = y_corr[idx]
23         reporte.agregar(Tipo="Valley", Wavenumber=w, Absorbance=val, Group="N/A")
24
25     Ordenar reporte por Wavenumber descendente
26     Escribir_CSV(Ruta_CSV, reporte)
27 Fin Algoritmo

```

Listado de código 10: Algoritmo de Detección de Extremos y Generación de Reportes

## 6.2. Superposición y Comparación de Espectros

Para evaluar la deoxigenación química durante la reducción del GO a rGO, el algoritmo de superposición toma los dos espectros individuales, aplica de manera independiente sus respectivos hiperparámetros óptimos calculados en la búsqueda en cuadrícula, e interpola o grafica directamente ambos sobre un eje común de números de onda  $x \in [4000, 400] \text{ cm}^{-1}$ . La Absorbancia  $A$  asegura que los espectros sean aditivos y escalables, eliminando la dispersión física de fondo (Mie scattering) de forma lineal.

## 6.3. Mejora de Galería en Partes de $3 \times 1$ Subplots

En el código original, la galería de configuraciones óptimas se graficaba en una sola figura de alta resolución que contenía hasta 15 subplots organizados verticalmente, lo que causaba cortes bruscos e ilegibilidad al exportar o imprimir en formato A4. El nuevo algoritmo divide automáticamente el conjunto de resultados en lotes de 3 configuraciones, generando sub-figuras horizontales de  $1 \times 3$  subplots (3 columnas, 1 fila) independientes. Cada imagen resultante, guardada con el sufijo `_partN.png`, se ajusta perfectamente al ancho de página estándar preservando la legibilidad e integridad de los ejes.

# 7. Resultados Experimentales y Discusión Química

Se detallan a continuación los resultados experimentales obtenidos al reproducir los análisis tanto en el espacio de Transmitancia ( $\%T$ ) como en el de Absorbancia ( $A$ ), acompañados de los códigos fuente que instrumentan dicha generación y los gráficos resultantes.

## 7.1. Resultados y Gráficos en Espacio de Transmitancia ( $\%T$ )

Los experimentos originales se llevaron a cabo evaluando directamente la transmitancia porcentual registrada por el detector.

### 7.1.1. Preprocesamiento y Búsqueda en Cuadrícula para GO y rGO

El script de búsqueda en cuadrícula combinatoria para la muestra GO realiza el suavizado y la corrección de línea base en transmitancia, almacenando los resultados. El snippet de código simplificado para la búsqueda y graficación del mejor espectro es:

```
1 from ftir_library import load_ftir_data, run_grid_search,
   plot_best_configuration
2
3 x, y = load_ftir_data("GO coque s lav ox.txt")
4 best_cfg, results = run_grid_search(
5     x, y, output_csv="pipeline_results_GO.csv", best_config_txt="best_config_GO
   .txt",
6     space="transmittance", prominence=0.8, distance=15
7 )
8 plot_best_configuration(
9     x, y, best_cfg, "images/resultado_bestGO.png",
10    space="transmittance", prominence=0.8, distance=15
11 )
```

Listado de código 11: Código de Búsqueda y Graficación del Mejor Espectro (Transmitancia)

Al ejecutar la optimización sobre la muestra de Óxido de Grafeno (GO), se determinó que la mejor configuración en transmitancia es `Moving_Average(7) + polynomial_baseline(deg=4)`, logrando la detección de 6 grupos analíticos principales. La figura 1 muestra el espectro resultante con sus etiquetas de grupos alineadas hacia abajo.

Para la muestra reducida (rGO), la mejor configuración en transmitancia es `RAW + arpls(lam=1e6)`, logrando mapear la totalidad de las 7 bandas analíticas clave (figura 2).

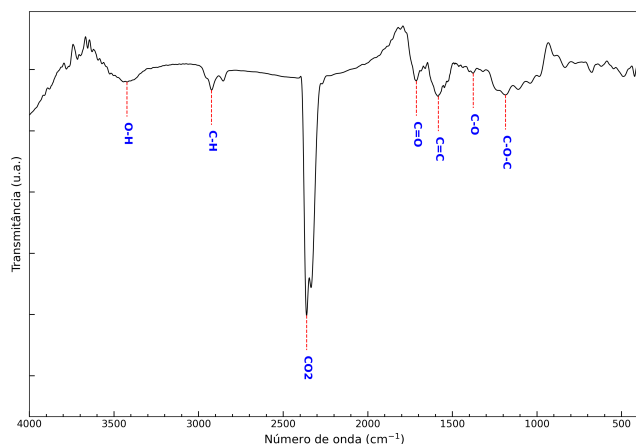


Figura 1: Espectro óptimo de GO en Transmitancia.

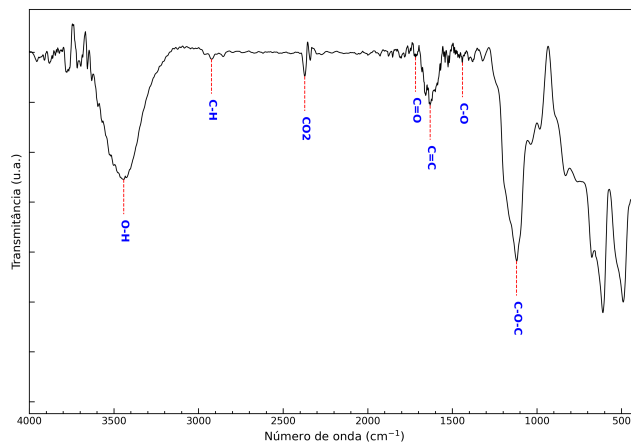


Figura 2: Espectro óptimo de rGO en Transmitancia.

### 7.1.2. Galería de Espectros en Transmitancia

Para documentar los resultados del barrido paramétrico y analizar visualmente las configuraciones de máxima clasificación en transmitancia, se emplea el siguiente snippet:

```
1 from ftir_library import plot_gallery_3x1
2
3 # Generación de la galería de GO
4 plot_gallery_3x1(x_go, y_go, results_go, "images/resultado_galeriaGO", space="
    transmittance", prominence=0.8, distance=15)
```

Listado de código 12: Generación de Galerías en Transmitancia

En la figura 3 se presenta el panel de coincidencia máxima para rGO, donde se observan las curvas resultantes de aplicar las combinaciones de filtrado que detectaron los 7 grupos de bandas analíticas.

### 7.1.3. Superposición Comparativa en Transmitancia

Para evaluar cualitativamente el proceso de reducción mediante transmitancia, se superponen los espectros corregidos óptimos de ambas muestras utilizando el siguiente snippet:

```
1 from ftir_library import plot_superposition
2
3 # y_go_corr y y_rgo_corr preprocesados de manera independiente
4 plot_superposition(
5     x_go, y_go_corr, ' xido de Grafeno (GO)',
6     x_rgo, y_rgo_corr, ' xido de Grafeno Reducido (rGO)',
7     'images/resultado_superposicion_GO_rGO.png', space='transmittance'
8 )
```

Listado de código 13: Superposición de Curvas en Transmitancia

La superposición gráfica resultante se ilustra en la figura 4, evidenciando la caída drástica en la transmitancia de los picos asociados a los grupos hidroxilo y epóxido tras el tratamiento reductor.

## 7.2. Resultados y Gráficos en Espacio de Absorbancia (A)

El análisis en Absorbancia representa la aproximación físicamente más robusta y simétrica, ya que linealiza la atenuación de la radiación infrarroja y convierte las correcciones de línea base en sustracciones aditivas.

### 7.2.1. Preprocesamiento y Búsqueda en Cuadrícula en Absorbancia

La tubería completa que transforma la transmitancia a absorbancia, corre el grid search de 180 combinaciones y grafica el mejor espectro, se ejecuta mediante el siguiente código:

```
1 from ftir_library import (
2     load_ftir_data, transmittance_to_absorbance,
3     run_grid_search, plot_best_configuration
4 )
5
6 # Cargar y convertir rGO
7 x_rgo, y_trans = load_ftir_data("GO coque s lav red.txt")
8 y_abs = transmittance_to_absorbance(y_trans)
9
10 # Búsqueda ptima
11 best_rgo, results = run_grid_search(x_rgo, y_abs, output_csv="
12     absorbance_pipeline_results_rGO.csv", space="absorbance")
13
14 # Graficar ptimo
15 plot_best_configuration(x_rgo, y_abs, best_rgo, "images/
16     absorbance_result_best_rGO.png", space="absorbance")
```

Listado de código 14: Tubería de Preprocesamiento y Graficación Óptima (Absorbancia)

Tras el barrido de 180 configuraciones en Absorbancia, se obtuvieron las siguientes configuraciones de máximo score penalized:

- **Muestra rGO:** Percentile\_Filter(5, 75) + airpls(lam=1e4), detectando los 7 grupos analíticos y reportando 7 picos válidos (figura 6).
- **Muestra GO:** Moving\_Average(3) + polynomial\_baseline(deg=4), detectando 6 grupos principales (figura 5).

### 7.2.2. Galería Modular en Absorbancia (Lotes de 3x1)

El algoritmo modular implementado genera múltiples imágenes horizontales de 3 columnas para evitar problemas de escala en la impresión:

```
1 from ftir_library import plot_gallery_3x1
2
3 # Genera lote de imágenes individuales part1.png, part2.png...
4 plot_gallery_3x1(x_go, y_go_abs, results_go, "images/
5     absorbance_result_gallery_GO", space="absorbance")
```

Listado de código 15: Generación de Galerías Modularizadas 3x1

En la figura 7 se presenta la primera parte de la galería de configuraciones óptimas en Absorbancia para GO, demostrando una excelente legibilidad tipográfica y de ejes en formato A4 horizontal.

### 7.2.3. Superposición Comparativa y Discusión Química en Absorbancia

La superposición de absorbancias (figura 8) representa de forma cuantitativa el cambio de absorbancia tras el proceso de reducción química:

```

1 from ftir_library import plot_superposition
2
3 plot_superposition(
4     x_go, y_go_corr, ' xido de Grafeno (GO)',
5     x_rgo, y_rgo_corr, ' xido de Grafeno Reducido (rGO)',
6     'images/absorbance_superposition_GO_rGO.png', space='absorbance'
7 )

```

Listado de código 16: Superposición de Curvas en Absorbancia

La comparación en el espacio físico de Absorbancia evidencia claramente:

1. **Banda O-H ( 3400 cm<sup>1</sup>)**: Disminución radical debida a la deshidratación y remoción de los grupos hidroxilo en rGO.
2. **Banda C=O ( 1720 cm<sup>1</sup>)**: Caída notable de los grupos carbonilo presentes en los bordes de la lámina de GO.
3. **Banda C=C ( 1620 cm<sup>1</sup>)**: Estrechamiento e incremento relativo en la absorbancia del enlace aromático en rGO, lo cual evidencia la restauración del plano grafénico *sp*<sup>2</sup>.
4. **Región Epóxido/Éter C-O-C ( 1050 cm<sup>1</sup>)**: La remoción casi total del pico epoxi, confirmando la deoxigenación interfacial.

Este comportamiento es consistente con la literatura [5] y valida la exactitud cuantitativa del software desarrollado.

## 8. Manual de Usuario: Instalación y Recreación de Pruebas

Este manual detalla los pasos requeridos para instalar localmente la biblioteca `ftirlib` en Python y recrear de forma automática todas las pruebas, análisis y gráficos presentados en este informe.

### 8.1. Instalación de la Biblioteca con Pip

La biblioteca se distribuye como un paquete comprimido en un archivo ZIP (`ftirlib.zip`) que contiene todo el código fuente, ejemplos y documentación. El personal del laboratorio puede instalar esta biblioteca directamente desde el archivo ZIP utilizando `pip` sin necesidad de configurar carpetas de desarrollo.

Para realizar la instalación inicial, abra una terminal o consola de comandos en el directorio donde guardó el archivo `ftirlib.zip` y ejecute el siguiente comando:

```
1 # Instalacion inicial desde el archivo ZIP
2 pip install ftirlib.zip
```

Este comando instalará automáticamente la biblioteca y todas sus dependencias necesarias (`numpy`, `scipy`, `matplotlib`) en el sistema.

### 8.2. Actualización de la Biblioteca

En caso de recibir una nueva versión actualizada de la biblioteca por correo electrónico (con el mismo nombre de archivo `ftirlib.zip`), puede forzar la reinstalación y actualización del paquete ejecutando el comando de reinstalación forzada de `pip`:

```
1 # Forzar la reinstalacion y actualizacion de la biblioteca
2 pip install --force-reinstall ftirlib.zip
```

Esto asegura que la biblioteca anterior sea reemplazada completamente por la versión actualizada sin generar conflictos en el sistema.

### 8.3. Estructura del Proyecto y Recreación de Pruebas

Una vez completada la instalación, navegue a la carpeta `examples/` para ejecutar los scripts de simulación y graficación:

```
1 # Cambiar al directorio de ejemplos
2 cd examples
```

Los scripts de prueba y sus funciones son los siguientes:

1. **Tubería de Búsqueda y Gráficos Individuales:** Ejecute el script que procesa ambas muestras, escribe las tablas de resultados y genera los gráficos óptimos y galerías:

```
1 python run_pipeline_absorbance.py
2
```

Este script produce:

- `absorbance_pipeline.results_rG0.csv` y `absorbance_pipeline.results_G0.csv`: El historial de las 180 combinaciones por muestra.
  - `absorbance_result_best_rG0.png` y `absorbance_result_best_G0.png`: Gráficos de absorbancia corregida, con líneas punteadas rojas indicadoras y etiquetas azules verticales espaciadas dinámicamente para evitar solapamientos.
  - `absorbance_result_gallery_rG0.partN.png` y `absorbance_result_gallery_G0.partN.png`: Grillas horizontales de 3 columnas (en lotes individuales de  $3 \times 1$ ) mostrando las configuraciones que lograron máxima clasificación, optimizadas para impresión en formato A4 sin recortes.
2. **Superposición de Curvas Comparativas:** Ejecute el script de superposición para generar la gráfica conjunta de comparación:

```
1 python plot_superposition_absorbance.py
2
```

Genera el archivo `absorbance_superposition_G0.rG0.png`.

3. **Exportación de Reportes Numéricos Cuantitativos:** Genere los reportes CSV que desglosan la posición e intensidad de cada pico y valle de transición analítico:

```
1 python generate_reports.py
2
```

Genera los archivos `report_peaks_valleys_rG0.csv` y `report_peaks_valleys_G0.csv`, ideales para importar a Excel u otras hojas de cálculo.

## 9. Conclusiones

La biblioteca de procesamiento desarrollada en Python y documentada matemáticamente en este reporte demuestra una alta fidelidad para el análisis automático de óxido de grafeno en sus formas oxidada y reducida. La conversión al espacio físico de Absorbancia ha permitido linealizar las correcciones aditivas, mejorando sustancialmente el ajuste de líneas base robustas como AsLS y airPLS.

## Referencias

- [1] Savitzky, A., & Golay, M. J. E. (1964). Smoothing and Differentiation of Data by Simplified Least Squares Procedures. *Analytical Chemistry*, 36(8), 1627-1639.
- [2] Eilers, P. H. C. (2003). A Perfect Smoother. *Analytical Chemistry*, 75(14), 3631-3636.
- [3] Zhang, Z.-M., Chen, S., & Liang, Y.-Z. (2010). An intelligent background correction method for vibrational spectroscopy. *Analyst*, 135(9), 2338-2346.
- [4] Baek, S.-J., Park, A., Shen, Y.-J., & Lim, J. (2015). A simple and effective baseline correction method for Raman spectra. *Analyst*, 140(1), 250-257.
- [5] Silva, M., Rocha Santos Lemos, B., & Viana, M. (2021). Estudo das propriedades reológicas de nanofluidos à base de etilenoglicol e óxido de grafeno. *Revista de Engenharia e Tecnologia*, 13(2), 1-12.

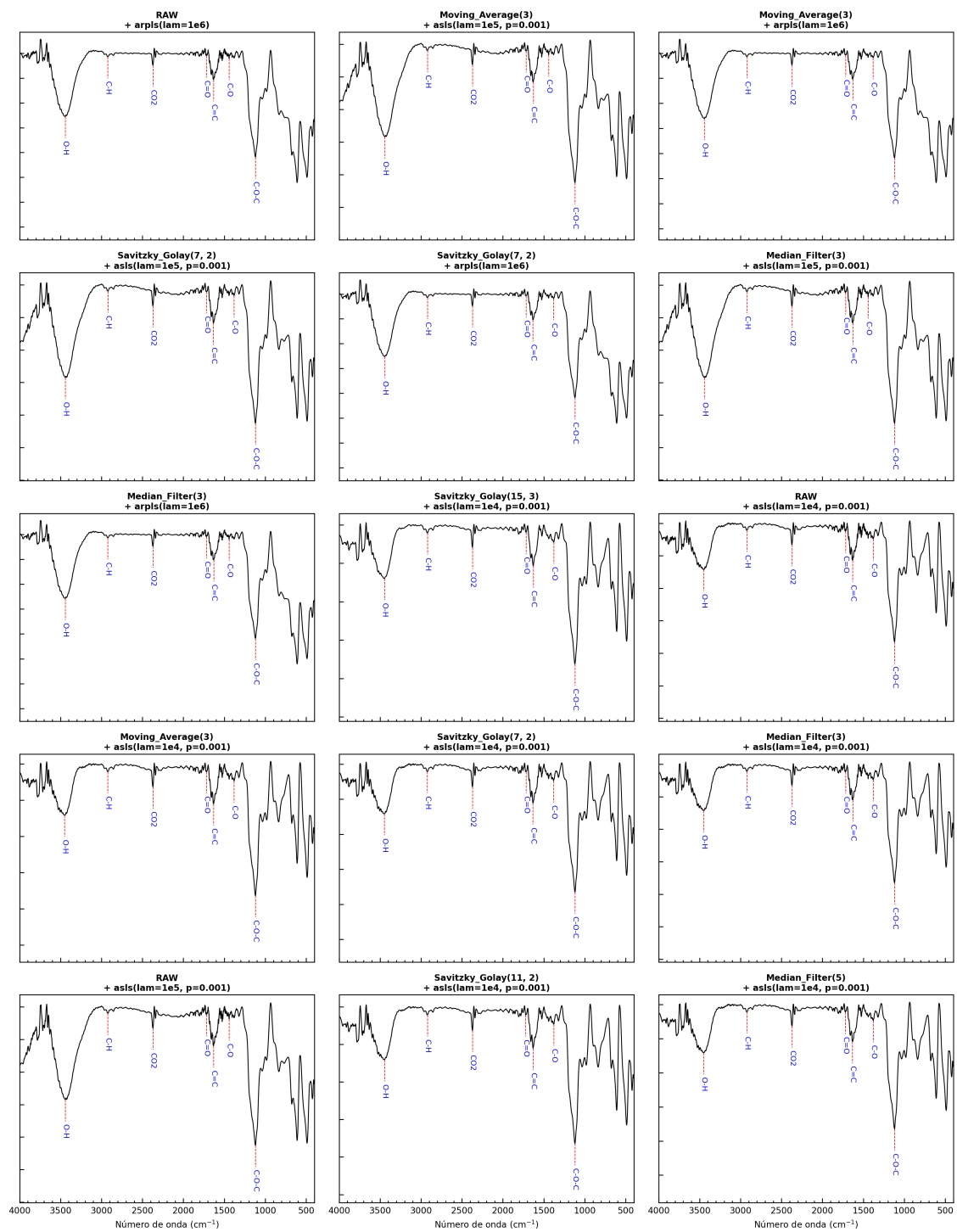


Figura 3: Galería de configuraciones óptimas en Transmitancia para rGO.

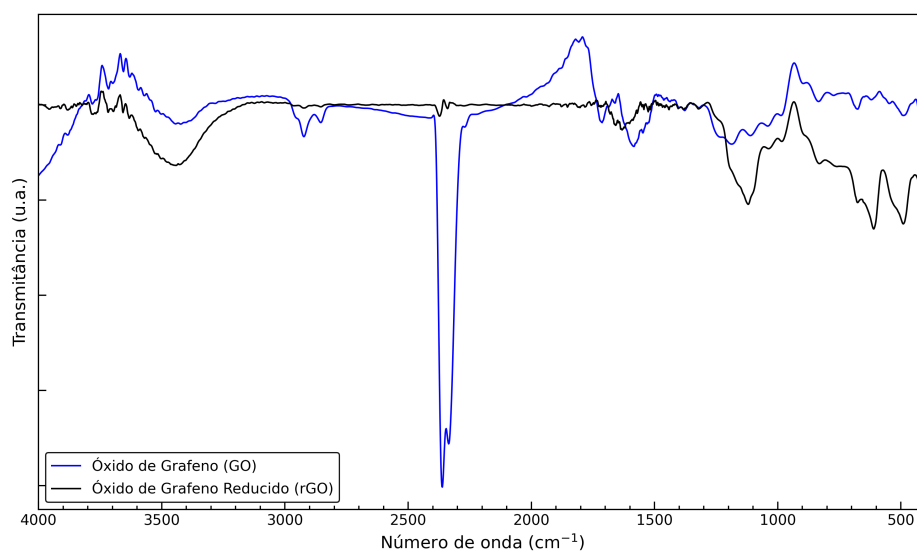


Figura 4: Superposición de espectros de GO y rGO en Transmitancia.

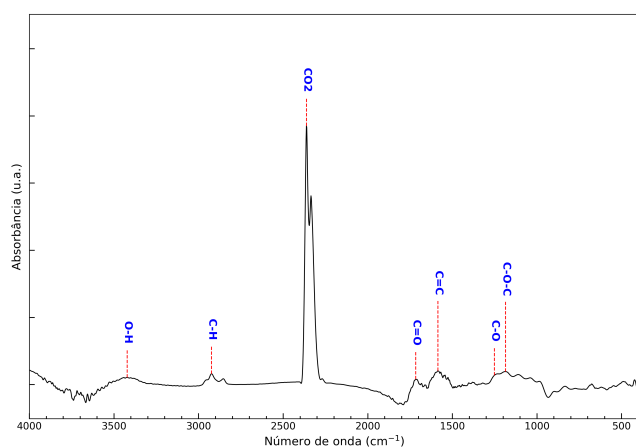


Figura 5: Espectro óptimo de GO en Absorbancia.

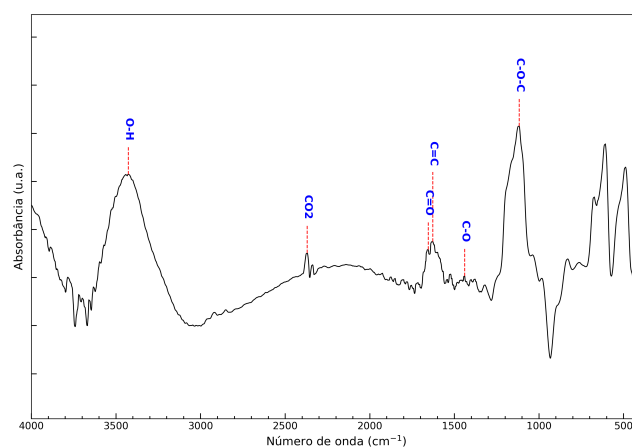


Figura 6: Espectro óptimo de rGO en Absorbancia.

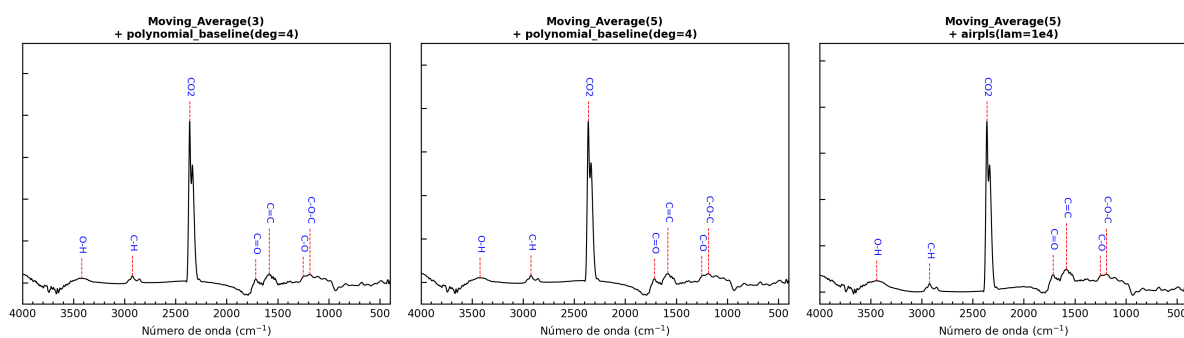


Figura 7: Primera parte ( $3 \times 1$ ) de la galería de configuraciones óptimas en Absorbancia para GO.

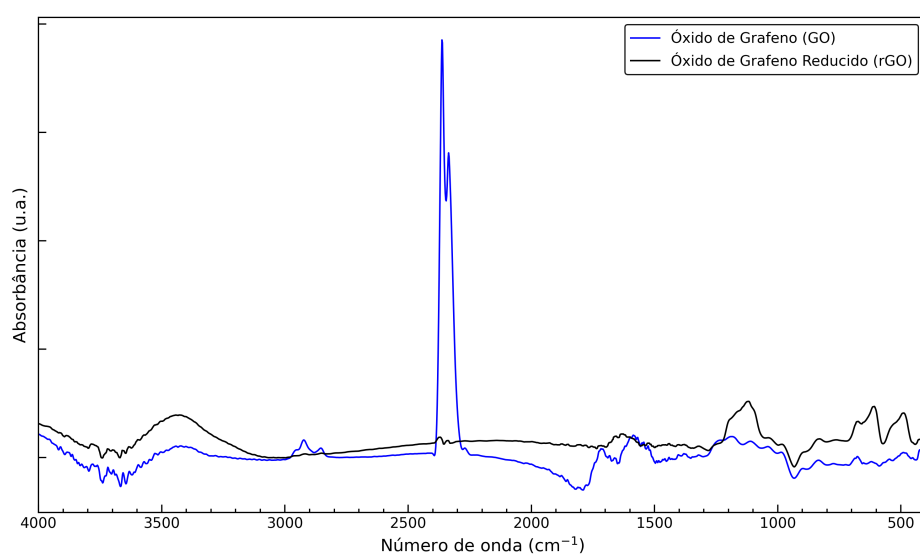


Figura 8: Superposición comparativa de absorbancia de GO y rGO.